Phase 5: Multi-Agent System with LangGraph

POC-07 — Inventory Management & Procurement System

Phase Weight: 20% | **Duration:** 5 working days | **Test Cases:** 25

---

1. Phase Overview

Build a LangGraph multi-agent pipeline that analyzes a product's inventory situation end-to-end: forecasting demand, recommending reorder quantities, coordinating with the best supplier, and auditing inventory health.

---

2. State Schema (multi_agent/state.py)

```
from typing import TypedDict, List, Dict, Any

class InventoryAnalysisState(TypedDict):
    product_id: int
    product_data: Dict[str, Any]           # Product + stock details
    demand_forecast: Dict[str, Any]        # Demand Forecaster output
    reorder_recommendation: Dict[str, Any] # Reorder Agent output
    supplier_quote: Dict[str, Any]         # Supplier Coordinator output
    audit_report: str                      # Inventory Auditor final report
    analysis_status: str                   # "analyzing" | "reorder_required" |
"healthy" | "complete" | "error"
    errors: List[str]
    messages: List[str]

def initial_state(product_id: int) -> InventoryAnalysisState:
    return InventoryAnalysisState(
        product_id=product_id, product_data={}, demand_forecast={},
        reorder_recommendation={}, supplier_quote={}, audit_report="",
        analysis_status="analyzing", errors=[], messages=[]
    )
```

---

3. Agents (multi_agent/agents.py)

```
import requests
import json
import structlog
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain.schema import HumanMessage
from langsmith import traceable

logger = structlog.get_logger()
```

```
BASE_URL = "http://localhost:8000/api/v1"
_llm = ChatGoogleGenerativeAI(model="gemini-2.0-flash", temperature=0.2)

def _safe_json(text: str) -> dict:
    try:
        start = text.find("{"); end = text.rfind("}") + 1
        return json.loads(text[start:end]) if start >= 0 and end > start else {}
    except Exception:
        return {}

@traceable(project_name="AI-Readiness-POC-07-P5")
def demand_forecaster(state: InventoryAnalysisState) -> InventoryAnalysisState:
    """Agent 1: Fetch product data and forecast demand."""
    product_id = state["product_id"]
    errors = list(state["errors"]); messages = list(state["messages"])
    product_data = {}

    try:
        r = requests.get(f"{BASE_URL}/products/{product_id}", timeout=10)
        r.raise_for_status()
        product_data = r.json()
    except Exception as e:
        errors.append(f"Product fetch error: {str(e)}")

    prompt = f"""Analyze product inventory data and forecast demand. Respond with
JSON only.
Product data: {json.dumps(product_data)}

JSON response:
{{
  "avg_daily_demand": <number or 0>,
  "demand_trend": "increasing|stable|decreasing|unknown",
  "days_of_stock_remaining": <number or 0>,
  "stockout_risk": "high|medium|low|none",
  "forecast_notes": "<one sentence>"
}}"""

    demand_forecast = {}
    try:
        resp = _llm.invoke([HumanMessage(content=prompt)])
        demand_forecast = _safe_json(resp.content) or {
            "avg_daily_demand": 0, "demand_trend": "unknown",
            "days_of_stock_remaining": 0, "stockout_risk": "unknown",
"forecast_notes": "Insufficient data."}
    except Exception as e:
        errors.append(f"Demand LLM error: {str(e)}")
        demand_forecast = {"avg_daily_demand": 0, "stockout_risk": "unknown"}

    messages.append(f"Demand Forecaster: risk=
{demand_forecast.get('stockout_risk')}, trend=
{demand_forecast.get('demand_trend')}")
    logger.info("agent_complete", poc_id="POC-07", phase="P5",
agent="demand_forecaster")
    return {**state, "product_data": product_data, "demand_forecast":
```

```
demand_forecast,
            "errors": errors, "messages": messages}

@traceable(project_name="AI-Readiness-POC-07-P5")
def reorder_agent(state: InventoryAnalysisState) -> InventoryAnalysisState:
    """Agent 2: Determine if reorder is needed and recommend quantity."""
    errors = list(state["errors"]); messages = list(state["messages"])

    prompt = f"""Determine if reorder is needed. Respond with JSON only.
Product: {json.dumps(state['product_data'])}
Demand forecast: {json.dumps(state['demand_forecast'])}

JSON response:
{{
  "reorder_required": <true|false>,
  "recommended_quantity": <number or 0>,
  "urgency": "immediate|within_3_days|within_week|not_required",
  "reason": "<explanation>"
}}"""

    reorder_recommendation = {}
    try:
        resp = _llm.invoke([HumanMessage(content=prompt)])
        reorder_recommendation = _safe_json(resp.content) or {
            "reorder_required": False, "recommended_quantity": 0,
            "urgency": "not_required", "reason": "Analysis unavailable."}
    except Exception as e:
        errors.append(f"Reorder LLM error: {str(e)}")
        reorder_recommendation = {"reorder_required": False, "urgency":
"not_required"}

    new_status = "reorder_required" if
reorder_recommendation.get("reorder_required") else state["analysis_status"]
    messages.append(f"Reorder Agent: required=
{reorder_recommendation.get('reorder_required')}, urgency=
{reorder_recommendation.get('urgency')}")
    logger.info("agent_complete", poc_id="POC-07", phase="P5",
agent="reorder_agent")
    return {**state, "reorder_recommendation": reorder_recommendation,
            "analysis_status": new_status, "errors": errors, "messages": messages}

@traceable(project_name="AI-Readiness-POC-07-P5")
def supplier_coordinator(state: InventoryAnalysisState) -> InventoryAnalysisState:
    """Agent 3: Find best supplier and generate quote for recommended quantity."""
    errors = list(state["errors"]); messages = list(state["messages"])
    supplier_quote = {}

    # Fetch supplier catalog if supplier_id available
    supplier_id = state["product_data"].get("supplier_id")
    catalog = []
    if supplier_id:
        try:
            r = requests.get(f"{BASE_URL}/suppliers/{supplier_id}/catalog",
timeout=10)
```

```
            if r.status_code == 200:
                catalog = r.json()
        except Exception as e:
            errors.append(f"Supplier fetch error: {str(e)}")

    prompt = f"""Generate a supplier quote. Respond with JSON only.
Product: {json.dumps(state['product_data'])}
Reorder recommendation: {json.dumps(state['reorder_recommendation'])}
Supplier catalog: {json.dumps(catalog[:5])}

JSON response:
{{
  "supplier_id": <number or null>,
  "quoted_unit_cost": <number or 0>,
  "total_order_cost": <number or 0>,
  "estimated_lead_time_days": <number or 7>,
  "quote_notes": "<recommendation>"
}}"""

    try:
        resp = _llm.invoke([HumanMessage(content=prompt)])
        supplier_quote = _safe_json(resp.content) or {
            "supplier_id": supplier_id, "quoted_unit_cost": 0,
            "total_order_cost": 0, "estimated_lead_time_days": 7, "quote_notes":
"Quote unavailable."}
    except Exception as e:
        errors.append(f"Supplier LLM error: {str(e)}")
        supplier_quote = {"supplier_id": None, "quoted_unit_cost": 0}

    messages.append(f"Supplier Coordinator: supplier=
{supplier_quote.get('supplier_id')}, cost=₹{supplier_quote.get('total_order_cost',
0):.0f}")
    logger.info("agent_complete", poc_id="POC-07", phase="P5",
agent="supplier_coordinator")
    return {**state, "supplier_quote": supplier_quote, "errors": errors,
"messages": messages}

@traceable(project_name="AI-Readiness-POC-07-P5")
def inventory_auditor(state: InventoryAnalysisState) -> InventoryAnalysisState:
    """Agent 4: Generate comprehensive inventory audit report."""
    messages = list(state["messages"])
    product = state["product_data"]
    forecast = state["demand_forecast"]
    reorder = state["reorder_recommendation"]
    quote = state["supplier_quote"]

    stock = product.get("stock_level", {})
    prompt = f"""Generate a concise inventory audit report. 3-4 sentences. Be
specific.
Product: {product.get('sku', 'Unknown')} — {product.get('name', '')}
Stock: {stock.get('quantity_available', 0)} available, reorder point:
{product.get('reorder_point', 0)}
Forecast: {forecast.get('days_of_stock_remaining', 0)} days remaining, risk:
{forecast.get('stockout_risk', 'unknown')}
```

```
Reorder: {'REQUIRED' if reorder.get('reorder_required') else 'Not required'} —
urgency: {reorder.get('urgency', 'N/A')}
Supplier quote: ₹{quote.get('total_order_cost', 0):.0f} for
{reorder.get('recommended_quantity', 0)} units"""

    audit_report = "Inventory audit complete."
    try:
        resp = _llm.invoke([HumanMessage(content=prompt)])
        audit_report = resp.content.strip()
    except Exception as e:
        audit_report = f"Audit error: {str(e)}"

    messages.append(f"Inventory Auditor: report generated ({len(audit_report)}
chars)")
    logger.info("agent_complete", poc_id="POC-07", phase="P5",
agent="inventory_auditor")
    return {**state, "audit_report": audit_report, "analysis_status": "complete",
"messages": messages}
```

---

4. Graph (multi_agent/graph.py)

```
from langgraph.graph import StateGraph, END
from multi_agent.state import InventoryAnalysisState, initial_state
from multi_agent.agents import demand_forecaster, reorder_agent,
supplier_coordinator, inventory_auditor
from langsmith import traceable

def should_skip_to_audit(state: InventoryAnalysisState) -> str:
    if len(state.get("errors", [])) >= 3 or state.get("analysis_status") ==
"error":
        return "inventory_auditor"
    return "reorder_agent"

def build_inventory_graph():
    graph = StateGraph(InventoryAnalysisState)
    graph.add_node("demand_forecaster", demand_forecaster)
    graph.add_node("reorder_agent", reorder_agent)
    graph.add_node("supplier_coordinator", supplier_coordinator)
    graph.add_node("inventory_auditor", inventory_auditor)
    graph.set_entry_point("demand_forecaster")
    graph.add_conditional_edges("demand_forecaster", should_skip_to_audit,
                                {"reorder_agent": "reorder_agent",
"inventory_auditor": "inventory_auditor"})
    graph.add_edge("reorder_agent", "supplier_coordinator")
    graph.add_edge("supplier_coordinator", "inventory_auditor")
    graph.add_edge("inventory_auditor", END)
    return graph.compile()

@traceable(project_name="AI-Readiness-POC-07-P5")
```

```
def analyze_product(product_id: int) -> InventoryAnalysisState:
    return build_inventory_graph().invoke(initial_state(product_id))
```

---

5. Submission Checklist

- [ ] `InventoryAnalysisState` TypedDict with 9 fields
- [ ] 4 agents: demand_forecaster, reorder_agent, supplier_coordinator, inventory_auditor
- [ ] Conditional edge after demand_forecaster (errors ≥ 3 → audit)
- [ ] `analyze_product(product_id)` as public entry
- [ ] `analysis_status == "complete"` after auditor
- [ ] LangSmith project "AI-Readiness-POC-07-P5"
- [ ] 25 test cases: ≥18 passing